ECMAScript Abort Protocol

A discussion on cooperative cancellation at the language level

The Gap

Cancellation is fundamental to async and long-running work.

The ecosystem has converged on `AbortController` / `AbortSignal`.

But these are **WHATWG DOM APIs**, not ECMAScript.

- TC39 cannot normatively reference Web APIs
- Not all JS environments implement them
- The language has no built-in concept of cancellation
- Yet the entire ecosystem depends on one

Goal

Define an **abort protocol** — a minimal set of requirements that any object can satisfy to participate in cooperative cancellation.

Designed so that `AbortSignal` can **retroactively satisfy it** with minimal or no changes to the Web platform.

Design Principles

Principle 1: Receiving a Signal

The core requirement: a way for a **consumer** to be told "stop what you're doing."

One-way communication:

Controller (decides to cancel) → **Consumer** (does cancellable work)

The consumer needs a way to register interest in being notified when abort occurs.

Principle 2: Synchronous & Immediate

Notification **must** be synchronous — same turn, no microtask/macrotask deferral.

- Consumer may be about to commit a side effect — async deferral means the side effect happens anyway
- Cancellation is time-sensitive — abort must fire *now*, not after the next promise settles
- Matches existing `AbortSignal` behavior

> An abort signal is an **interrupt**, not a suggestion.

Principle 3 & 4: State Checks

Pre-check: "Has abort already occurred?"

Before starting work, check if the signal is already aborted.

```
if (signal.aborted) throw signal.reason;
```

Post-check: "Did abort occur while I was busy?"

After uninterruptible synchronous work, check again.

```
1. Check state      → not aborted
2. Register listener → begin work
3. Synchronous computation (can't respond mid-work)
4. Check state again → may now be aborted
```

Same mechanism (boolean state), two distinct use cases.

Principle 5: Reason Travels With the Signal

An abort is not just "stop" — it's **"stop, and here's why."**

The signal carries a **reason** (any type, conventionally an `Error`):

- Why: timeout, user cancellation, parent task failure, resource limit
- Can be rethrown or wrapped by the consumer
- Available via both notification callback and state inspection
- Set at the moment of abort, stable thereafter

Principle 6: One-Way, One-Time, Irreversible

A signal transitions from "not aborted" → "aborted" **exactly once**.

- No un-abort, re-abort, or state oscillation
- Monotonic: once `true`, always `true`
- No complex state machine — just a boolean with a one-time transition

Principle 7: Receiving, Not Sending

The protocol defines **what a signal looks like** — the consumption interface.

How abort is *triggered* is a separate concern.

- Multiple consumers can share one signal
- Trigger mechanism varies (explicit call, timeout, parent cancellation)
- Protocol only standardizes what consumers depend on
- `AbortController` is one possible controller; others can exist

Principle 8: Unsubscription

A consumer must be able to say **"I no longer need to be told."**

- Consumer interest is bounded by the lifetime of the work
- Without unsubscription:
  - Long-lived signals leak dead references
  - Composed signals cannot clean up after one input fires
  - Stale listeners fire on closed resources
- Unsubscription must be **synchronous and immediate**
  - After unsubscription, callback must never be invoked again

Principle 9: Multi-Consumer

A single signal is routinely shared across multiple independent consumers.

```
fetch(url, { signal })    ← fetch implementation
  ↑                        ← timeout wrapper
  ↑                        ← application cleanup logic
  ↑                        ← composed signal monitoring
```

- Each consumer subscribes and unsubscribes independently
- Single-consumer models (`onabort` property) create silent-replacement bugs
- `AbortSignal` is ambiguous: `addEventListener` (multi) vs `onabort` (single)

Principle 10: Multiple Sources

Consumers often need **multiple cancellation sources** simultaneously.

- Per-request timeout **and** server shutdown signal
- User cancel **and** parent task abort
- Upstream deadline **and** internal resource limit

The protocol does not mandate a composition primitive, but its shape **must enable** building `any()`-style combinators.

Only the "any" combinator (first-to-fire) has demonstrated practical need.

Principle 11: The "Never Aborts" Signal

A signal that **will never abort** — the symmetric degenerate case.

- Eliminates pervasive null-checking when no cancellation is desired
- Consumers interact uniformly — no `if (signal?.aborted)` everywhere
- **Discards registrations entirely** — zero-cost, no leak path
- Protocol shape should make this trivially constructible

Principle 12: Abort Is Always Explicit

Abort must be a **deliberate, explicit act**. Never a side effect.

- **Disposal ≠ abort.** `using` / `Symbol.dispose` means "done managing this" — not "cancel all work"
- **GC ≠ abort.** Signal becoming unreachable is not an abort
- **Unsubscription ≠ abort.** Removing your own listener doesn't interrupt other consumers

Abort has consequences (interrupts work, propagates exceptions). It must never happen as a surprise.

Principle 13: Late Listeners Are Ignored

Registering a listener on an **already-aborted** signal: silently ignored.

```
1. Check .aborted → if true, handle immediately
2. Register listener
3. Begin work
```

- Consumers check state first (Principle 3), then register
- Auto-firing late listeners creates hidden timing-dependent code paths
- Conflates state checking and event subscription
- Differs from `AbortSignal` (which fires late listeners synchronously)

Protocol Shape

Constraints

The protocol **cannot** normatively reference Web platform APIs:

Unavailable	Why
`EventTarget`	Web API — subscription model `AbortSignal` uses
`AbortSignal`	WHATWG DOM, not ECMAScript
`AbortController`	WHATWG DOM
`Event`	WHATWG DOM

Not merely bureaucratic — the protocol must work in **all** ECMAScript environments: embedded engines, serverless runtimes, non-browser hosts.

What the Protocol Must Express

Four things:

#	Capability	Shape
1	**State**: has abort occurred?	Synchronous boolean read
2	**Reason**: why?	Synchronous read of arbitrary value
3	**Subscribe**: notify me	Register a callback
4	**Unsubscribe**: stop notifying me	Remove a registration

(1) and (2) are straightforward properties.

The design question is (3) and (4): **the subscription mechanism.**

The Subscription Problem

ECMAScript has no general-purpose event subscription primitive.

Pattern	Available in ES?	Multi-consumer?	Notes
`addEventListener` / `removeEventListener`	No (Web API)	Yes	Requires function identity for removal
`.on` / `.off`	No (Node.js)	Yes	Same shape, also not ES
`signal.onabort = fn`	Yes	**No**	Last-writer-wins
`subscribe()` → unsubscribe fn	**Yes**	**Yes**	Used by RxJS, MobX, Redux, Solid, Svelte

Subscribe-Returns-Unsubscribe

```
const unsubscribe = signal.subscribe(fn);
// later:
unsubscribe();
```

- **Independent unsubscription** — each call returns a unique handle
- **No reference identity** — no need to keep the original `fn` reference
- **Multi-consumer** — each subscription is independent
- **Composable with `using`** — unsubscribe can be `Symbol.dispose`
- **Pure ECMAScript** — no Web API dependency

Why Not Observable?

The TC39 Observable proposal (Stage 1) provides subscription with teardown.

- Observable models **multi-value async streams** — abort is a single event
- Conceptual mismatch: next/error/complete lifecycle is unnecessary
- **Dependency ordering** — abort protocol would be blocked on Observable
- **Circular dependency** — Observable itself is a likely consumer of abort signaling
- Subscribe-returns-unsubscribe is *compatible* with Observable as a layer on top

Design Options

Option A: Well-Known Symbol Protocol

Like `Symbol.iterator` — a Symbol method returns a protocol object.

```
const cancel = signal[Symbol.cancelation]();
cancel.aborted;              // boolean
cancel.reason;               // any
cancel.subscribe(fn);        // → unsubscribe function
```

AbortSignal retrofit: WHATWG adds `[Symbol.cancelation]()` returning a wrapper.

```
AbortSignal.prototype[Symbol.cancelation] = function () {
  const signal = this;
  return {
    get aborted() { return signal.aborted; },
    get reason() { return signal.reason; },
    subscribe(fn) {
      signal.addEventListener('abort', fn, { once: true });
      return () => signal.removeEventListener('abort', fn);
    }
  };
};
```

Option A: Observations

- **Two-step access.** Consumer calls `[Symbol.cancelation]()` then uses the result. Unlike iterators (where iterable vs iterator is meaningful), cancel signals have singular state — no analogous reason to separate source from protocol object.

- **Protocol object identity.** Source can return `this` (as arrays do for `Symbol.iterator`). If so, collapses the distinction with Option D.

- **Symbol-based identification.** Well-known Symbol cannot collide with existing string-named properties. The Symbol method produces the protocol object, so calling it inherently yields an object with the expected shape.

Option B: Duck Typing

Like thenables — any object with the right shape is treated as a signal.

```
// Anything with .aborted, .reason, and .subscribe
if (signal.aborted) throw signal.reason;
const unsubscribe = signal.subscribe(fn);
```

AbortSignal retrofit: Add one method. Existing `.aborted` and `.reason` already conform.

```
AbortSignal.prototype.subscribe = function (fn) {
  this.addEventListener('abort', fn, { once: true });
  return () => this.removeEventListener('abort', fn);
};
```

Option B: Observations

- **Direct access.** No `[Symbol.cancelation]()` call. Consumers interact directly with properties and methods.
- `AbortSignal` **retrofit.** One method addition. Existing `.aborted` and `.reason` already conform. No wrapper objects.
- **No formal identification.** An object with an unrelated `.aborted` property and a `.subscribe` method would satisfy the protocol shape. Collision risk depends on the specificity of the property names chosen.
- **Name collision.** `subscribe` is used by other patterns (Observable, pub/sub). A more specific name may conflict with existing properties.

Option C: Concrete Built-in Class

New `CancelSignal` / `CancelController` classes in ECMAScript.

```
const controller = new CancelController();
doWork(controller.signal);
controller.cancel(new Error('timeout'));
```

AbortSignal retrofit requires one of:

- Spec-level coordination between TC39 and WHATWG
- Protocol Symbol fallback (collapses to Option A/D)
- Bridge utility: `CancelSignal.from(abortSignal)`

Option C: Observations

- **API surface.** Static methods (`CancelSignal.any()`, `.never()`, `.from()`) and `instanceof` checking.
- **Specification footprint.** Defines both a signal class and a controller class, with constructors, prototypes, and internal slots.
- `AbortSignal` **retrofit.** `AbortSignal` cannot retroactively extend `CancelSignal`. Requires duck typing, a shared protocol Symbol, or spec-level coordination.
- **Coexistence with** `AbortSignal`. Two signal types serving the same purpose. APIs must decide which to accept, or bridge between them.
- `instanceof` **and realms.** Identification within a single realm but not across realm boundaries.

Option D: Symbol + Direct Conformance

Hybrid of A and B. Symbol for identification, properties directly on the object.

```
// Consumers use properties directly:
if (signal.aborted) throw signal.reason;
const unsubscribe = signal.subscribe(fn);

// Symbol serves as a brand for identification:
if (signal && !signal[Symbol.cancelation]) {
  throw new TypeError('not a cancel signal');
}
```

AbortSignal retrofit: Add Symbol brand + `.subscribe`.

```
AbortSignal.prototype[Symbol.cancelation] = function () { return this; };
AbortSignal.prototype.subscribe = function (fn) {
  this.addEventListener('abort', fn, { once: true });
  return () => this.removeEventListener('abort', fn);
};
```

Option D: Observations

- **Direct access with Symbol identification.** Consumers interact with properties directly. Symbol is present for identification but not required for access.
- **The Symbol is a brand, not a factory.** Unlike `Symbol.iterator`, `[Symbol.cancelation]()` always returns `this`. Cancel signal state is singular.
- `AbortSignal` **retrofit.** Adds two things: the Symbol and `.subscribe`. Existing `.aborted` and `.reason` already conform. No wrapper objects.
- **Disconnected identification.** The Symbol asserts conformance, but consumers access named properties. Symbol's presence does not guarantee those properties exist or are correct. Consumer code that wants to be robust must still verify properties regardless of Symbol presence.

Comparison: AbortSignal Retrofit

Option	Changes to AbortSignal	Wrapper?
A: Symbol protocol	Add `[Symbol.cancelation]()`	Yes — returns new object
B: Duck typing	Add `.subscribe()`	No
C: Concrete class	Spec coordination or bridge	Varies
D: Symbol + direct	Add `[Symbol.cancelation]()` + `.subscribe()`	No

Never-Aborts Sentinel

All options support a "never aborts" signal:

```
// Option A
const neverAborts = {
  [Symbol.cancelation]() {
    return { aborted: false, reason: undefined, subscribe(_fn) { return () => {}; } };
  }
};

// Option B
const neverAborts = Object.freeze({
  aborted: false, reason: undefined, subscribe(_fn) { return () => {}; }
});

// Option C
const neverAborts = CancelSignal.never();

// Option D
const neverAborts = Object.freeze({
  aborted: false, reason: undefined, subscribe(_fn) { return () => {}; },
  [Symbol.cancelation]() { return this; }
});
```

Composition: `any()`

All protocol options enable building `any()` in userland:

```
function any(signals) {
  // Pre-check: any input already aborted?
  for (const s of signals) {
    if (s.aborted) return alreadyAborted(s.reason);
  }

  // Subscribe to all, fire on first trigger
  const unsubscribes = [];
  for (const s of signals) {
    const unsub = s.subscribe((reason) => {
      // Unsubscribe from remaining inputs
      for (const u of unsubscribes) u();
      // Notify composite subscribers
      notifyAll(reason);
    });
    unsubscribes.push(unsub);
  }

  return compositeSignal;
}
```

Open Design Questions

Questions for Committee Discussion

1. **Protocol identification:** Symbol-based, duck typing, or concrete class?

2. **Protocol object shape:** Subscribe method directly on the signal, or returned from a Symbol method as a separate object?

3. **Disposable unsubscribe?** Should the returned unsubscribe handle have `Symbol.dispose` for `using` integration?

Relationship to Existing Work

	Relationship
AbortController / AbortSignal	Protocol should be designed so AbortSignal can retroactively satisfy it
Explicit Resource Management	Disposal ≠ abort (Principle 12). Unsubscribe handle is a candidate for `Symbol.dispose`
Observable	Likely consumer of abort protocol. Compatible, not dependent
Error.cause	Abort reason typically propagated as error cause
Error.isError	Cross-realm identification faces similar challenges

Discussion

What are your thoughts on the design principles and protocol shape?